CampusPath — Updated PostgreSQL Database Schema (Sprint 2)

Database: `campuspath`
DBMS: PostgreSQL
Hosting: AWS RDS
Backend: Java Spring Boot
Authentication: AWS Cognito

1. Existing Sprint 1 Tables

`users`

Stores registered students and connects their Cognito identities to PostgreSQL.

Previously established fields:

- `user_id` — INTEGER, Primary Key (confirm exact name)
- `email` — VARCHAR
- `full_name` — VARCHAR
- `cognito_sub` — VARCHAR, Cognito user identifier
- `created_at` — TIMESTAMP/TIMESTAMPTZ

The existing Sprint 1 schema should be preserved. The UI also expects a student ID (`euid`) and university information in the user profile response.

`universities`

Stores registered universities.

Previously established fields:

- `id` — INTEGER, Primary Key
- `name` — VARCHAR
- `city` — VARCHAR
- `state` — VARCHAR
- `country` — VARCHAR

`degrees`

Stores degree programs.

Previously established fields:

- `id` — INTEGER, Primary Key
- `degree_name` — VARCHAR
- `degree_level` — VARCHAR

`student_degrees`

Links students with their degree programs.

Previously established fields:

- `id` — INTEGER, Primary Key
- `user_id` — INTEGER, Foreign Key
- `degree_id` — INTEGER, Foreign Key
- `start_term` — VARCHAR
- `status` — VARCHAR

Note: These Sprint 1 fields are based on our earlier design, not a fresh database export. Verify the actual column names and types using the database metadata before creating Java entities.

---

2. Sprint 2 Tables

`user_settings` — Existing, retained

- `user_id` — INTEGER, references users
- `timezone` — VARCHAR
- `theme` — VARCHAR
- `notifications_enabled` — BOOLEAN
- `email_reminders_enabled` — BOOLEAN
- `default_reminder_minutes` — INTEGER
- `updated_at` — TIMESTAMPTZ (if the optional migration was executed)

Stores application preferences, notifications, appearance, and reminder settings.

`schedule_preferences` — New

- `preference_id` — BIGINT, Primary Key
- `user_id` — INTEGER, Foreign Key → users
- `earliest_class_time` — TIME
- `latest_class_time` — TIME
- `max_credits` — INTEGER, range 3–21
- `preferred_days` — SMALLINT[]
- `minimize_gaps` — BOOLEAN
- `minimize_campus_days` — BOOLEAN

One preferences record per student.

Weekday convention: Monday = 1, Tuesday = 2, Wednesday = 3, Thursday = 4, Friday = 5, Saturday = 6, Sunday = 7.

The UI sends weekday strings such as `["MON", "WED", "FRI"]`. The backend should convert these to `[1, 3, 5]` when saving and reverse the conversion when retrieving preferences.

`commitments` — New

- `commitment_id` — BIGINT, Primary Key
- `user_id` — INTEGER, Foreign Key → users
- `title` — VARCHAR(160)
- `commitment_type` — VARCHAR(30)
- `day_of_week` — SMALLINT
- `start_time` — TIME
- `end_time` — TIME
- `start_date` — DATE
- `end_date` — DATE
- `color` — VARCHAR(7)

Stores recurring weekly commitments.

Supported categories include `class`, `work`, and `personal`. The database also permits `extracurricular` and `other`.

End time must be later than start time. End date must be on or after start date.

`calendar_events` — New

- `event_id` — BIGINT, Primary Key

- `user_id` — INTEGER, Foreign Key → users
- `title` — VARCHAR(160)
- `event_type` — VARCHAR(30)
- `event_date` — DATE
- `start_time` — TIME
- `end_time` — TIME
- `location` — VARCHAR(180)
- `notes` — VARCHAR(500)

Stores one-time calendar events.

Supported categories: `class`, `work`, `personal`.

End time must be later than start time.

`academic_terms` — New

- `term_id` — BIGINT, Primary Key
- `user_id` — INTEGER, Foreign Key → users
- `term_name` — VARCHAR(100)
- `start_date` — DATE
- `end_date` — DATE

Stores academic terms created by students.

Each student can create multiple terms, but cannot reuse the same term name.

`generated_schedules` — New

- `schedule_id` — BIGINT, Primary Key
- `user_id` — INTEGER, Foreign Key → users
- `term_id` — BIGINT, Foreign Key → academic_terms
- `schedule_name` — VARCHAR(120)
- `score` — NUMERIC(6,2), nullable
- `is_selected` — BOOLEAN
- `created_at` — TIMESTAMPTZ

Stores saved schedule options associated with academic terms.

Each student can have only one selected schedule per academic term, enforced by a partial unique index.

Actual class timetable generation is not part of the current Sprint 2 implementation.

---

3. Database Relationships

- `users` → `user_settings`: One-to-one
- `users` → `schedule_preferences`: One-to-one
- `users` → `commitments`: One-to-many
- `users` → `calendar_events`: One-to-many
- `users` → `academic_terms`: One-to-many
- `users` → `generated_schedules`: One-to-many
- `academic_terms` → `generated_schedules`: One-to-many

All student-specific queries must be scoped to the authenticated user.

4. Frontend JSON Mapping

PostgreSQL uses snake_case, while the JavaScript UI expects camelCase.

Database Column	Frontend JSON Property
`commitment_id`	`id`
`event_id`	`id`
`term_id`	`id`
`schedule_id`	`id`
`term_name`	`name`
`event_type`	`eventType`
`event_date`	`eventDate`
`commitment_type`	`commitmentType`
`day_of_week`	`dayOfWeek`
`start_time`	`startTime`

`end_time`	`endTime`
`start_date`	`startDate`
`end_date`	`endDate`
`schedule_name`	`scheduleName`
`is_selected`	`isSelected`
`created_at`	`createdAt`

The frontend sends an authenticated Cognito token with requests. The backend should validate that token, resolve the user's PostgreSQL record through `cognito_sub`, and perform operations using that user's ID.

Do not accept an arbitrary user ID from the browser for these operations.

5. Implementation Notes

- Reuse the existing PostgreSQL connection to AWS RDS.
- Preserve all Sprint 1 authentication functionality.
- Password changes should use AWS Cognito rather than directly updating PostgreSQL.
- Use Spring Boot DTOs to match the frontend's expected JSON properties.
- Restrict all database reads, updates, and deletions to the authenticated user.
- For selecting generated schedules, deselect any previously selected option for the same student and term within a database transaction.
- The database currently contains no actual student schedules; this is expected.
- The schema supports settings, recurring commitments, one-time events, academic terms, and saved schedule options.
- Check the database's actual metadata for exact keys, types, defaults, and constraints before implementing the Java entities.